

Staff Detection from Overhead Video

AI Evaluation | Solution documentation | 9 September 2026

1. Objective and approach

Identify video frames containing a staff member and report their image-plane XY position. The evaluation defines staff by a visible name tag. This prototype fine-tunes a pretrained YOLO26n detector on a single Staff class, then uses ByteTrack to associate detections across frames. It learns from annotated examples rather than explicitly matching or reading the badge; reliable badge-based generalization therefore remains a validation requirement.

2. Data preparation and model training

The supplied sample.mp4 contains 1,341 frames at 25 FPS (960 x 720 pixels). dataset_create.py samples every fifth frame, producing 269 images. Exported NDJSON annotations are converted into normalized YOLO labels by parse_ndjson.py. The current dataset contains 215 training images and 54 validation images. There are 48 annotated staff boxes in training and 11 in validation; the remaining images have empty labels and provide background examples.

staff_detector.py fine-tunes yolo26n.pt for 50 epochs at an input size of 640 pixels, batch size 4, using GPU device 0 and two data-loading workers. data.yaml defines the Staff class and dataset paths. The saved run provides best.pt and last.pt; inference scripts load best.pt from the run folder ending 04-32-58_e50_b4.

3. Detection, tracking and coordinates

OpenCV reads the video sequentially. Each frame is passed to model.track with persist=True and custom_tracker.yaml. ByteTrack uses high/new-track thresholds of 0.6, a low threshold of 0.1, a matching threshold of 0.6, a track buffer of 60, and score fusion enabled. These settings retain lost-track state temporarily but do not guarantee continuous IDs or detections during occlusion.

The current tracking.py logs results only when the model returns tracking IDs. A positive frame contains at least one returned track. The XY location is the integer bounding-box centre: $x = (x1 + x2) / 2$, $y = (y1 + y2) / 2$. Coordinates are pixels with origin at the image top-left; x increases rightward and y downward. They are not calibrated floor or 3D coordinates.

INPUT	MODEL	ASSOCIATION	OUTPUT
Video frames	YOLO26n / Staff	ByteTrack	Frame + ID + XY

Outputs: frame/staff_frame.txt lists positive frame numbers, IDs and XY coordinates; frame/frame_XXXX.jpg contains annotated images. Frame numbering starts at 1. Frames without returned IDs are omitted from the text log. An assigned ID is a tracking result, not independent verification that a name tag is present.

Results, limitations and reproduction

4. Recorded validation results

The final epoch of the saved training run reports the following box-detection metrics. These are final-epoch values, not a separate evaluation of the exported best.pt checkpoint or frame-level staff-presence accuracy.

Precision	Recall	mAP @ IoU 0.50	mAP @ IoU 0.50:0.95
99.41%	90.91%	90.50%	47.10%



Figure 1. Existing annotated output, frame 817: a Staff detection with track ID 12 and centre coordinate (524, 568). This illustrates output formatting; the displayed confidence is not an accuracy measurement or proof of correct identity.

5. Limitations and next validation steps

The split comes from one video: 52 of 54 validation images have an immediately adjacent sampled image in training. Similar appearance and background can inflate validation results, and only 11 validation staff boxes are available. An independent video or a temporally separated holdout is needed before claiming performance on unseen footage. The detector may learn clothing, person or scene cues instead of the tag itself.

Occlusion, tiny badges, rotation and lighting changes can cause missed detections, false positives or ID changes. Measure frame-level precision/recall against manually labelled presence intervals, and assess ID continuity and XY error separately. Future work should add diverse tagged/untagged examples, improve temporal continuity and record explicit negative frames. Runtime FPS has not been benchmarked for this documentation.

6. Running the prototype

From the project root, activate the environment with `.\yolo\Scripts\Activate.ps1` and install dependencies using `python -m pip install -r requirements.txt` if needed. Run `python tracking.py` for IDs, coordinates and frame images, or `python test_inteference.py` for an annotated video. Training is available through `python staff_detector.py`; it is unnecessary when using existing weights.

For another test video, update the hardcoded project/model/video paths and select a compatible device (the current scripts request GPU 0). Run from the project root so the tracker configuration resolves. Tracking overwrites its text log and matching frame-image filenames. `model_export.py` exports the trained weights to ONNX; exported-model parity has not been assessed.

Evidence: AI Evaluation Test.pdf; current Python scripts and YAML files; dataset labels; saved run results.csv (epoch 50); existing frame_0817.jpg. No new training or inference was performed to prepare this document.